HST

MAKING ELECTRONIC TRANSACTIONS **SECURE**

Issuer SDK
Push Provisioning

APPLE PAY

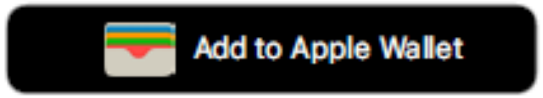
Mayo • 2023

HST
MAKING ELECTRONIC TRANSACTIONS SECURE

OBJETIVO DE ESTE DOCUMENTO

El objetivo de este documento es presentar una descripción de la solución de HST para Push Provisioning en Apple Pay.

HST Push Provisioning permite el envío de tarjetas a Apple Wallet, directamente desde el aplicativo del Emisor, sin fricción para el usuario final.



Además de la funcionalidad de Push Provisioning, el SDK participa del proceso de In-App Verification, donde se utiliza la App del emisor para autenticar al tarjetahabiente que ingresó los datos de tarjeta de forma manual en Apple Pay

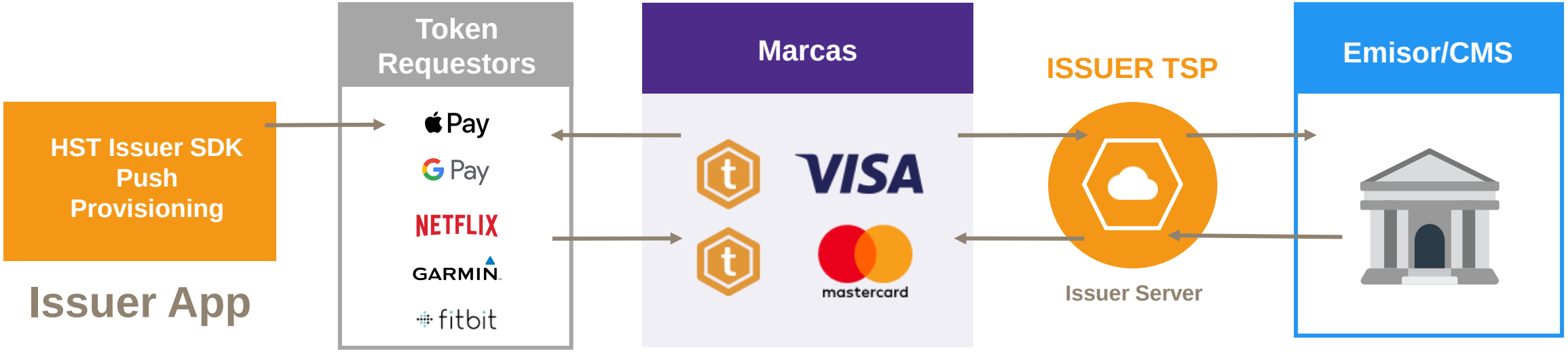
Requerimientos de Apple Pay

1 In-App Provisioning

2 In-App Verification

1 In-App Provisioning

Ecosistema de Tokenización



Ventajas para

TARJETAHABIENTES

Sus tarjetas son agregadas en Apple Pay directamente del aplicativo iOS del banco con un clic

No hay necesidad de ingresar los datos de la tarjeta manualmente o hacer verificación de identidad

Todo el proceso es realizado desde el aplicativo del banco, de forma rápida, segura y fácil

Ventajas para **EMISORES**

Incrementar el uso de la aplicación del banco al brindar la posibilidad de agregar las tarjetas a Apple Pay o a un Apple Watch desde un dispositivo iOS

Integrarse con Apple Pay a través del SDK de HST, reduciendo la complejidad de varios procesos

Cumplir con los requerimientos exigidos por Apple, que son cubiertos por el SDK de HST

Proveer a sus tarjetahabientes la posibilidad de hacer pagos NFC en dispositivos iOS mediante Apple Pay

Actividades del Proyecto Apple Pay

EMISORES

Para implementar la funcionalidad, el Emisor debe:

- Realizar la apertura de un proyecto de certificación con Apple y la marca para realizar el Onboarding y habilitar las tarjetas en Apple Pay.
- La aplicación del emisor debe soportar versión 9 o superiores de iOS.
- Seguir los *Guidelines* de Apple con respecto a la User Interface (UI) y a las funcionalidades para la Certificación especificadas en el manual *Apple Pay Card Issuer Functional Requirements* (v 3.3 o superior).
- Solicitar permisos de Apple para integrarse en su portal.
- Recibir un certificado de HST utilizado para la criptografía del PAN (opción C) o implementar las APIs necesarias para integración con el Issuer Server de HST (opciones A y B)
- Elegir un laboratorio para la Certificación con Apple (UL/FIME).
- Para VTS, las pruebas deberán ser realizadas directamente en entorno productivo. Sin embargo, MDES ya facilita un entorno de *testing*.
- Realizar una etapa de *Friends & Family* antes del despliegue masivo.

Principales Requisitos de Apple

EMISORES Issuer App

Para implementar la funcionalidad, el Emisor debe:

- Implementar la funcionalidad de In-App Provisioning (“Add to Apple Pay”).
- Implementar la funcionalidad de In-App Verification.
  Este requisito de Apple en general no es mandatorio, depende de la región del emisor.
- Seguir los *Guidelines* de UX definidos por Apple

Principales Requisitos de Apple

EMISORES I-TSP

Para implementar la funcionalidad, el Emisor debe:

- Controlar el número máximo de dispositivos que digitalizaron una misma tarjeta.
- Bloquear por 24 horas la digitalización de una tarjeta después de 5 intentos consecutivos con CVV2 inválido y/o fecha de expiración invalida.
- Soportar los flujos verde, amarillo y naranja.
- Notificar al cliente a respecto de una tarjeta digitalizada y no activada después de 24 horas.

Principales Requisitos de Apple

EMISORES Autorizador

Para implementar la funcionalidad, el Emisor debe:

- Notificar al cliente respecto de una tarjeta que no transaccionó después de 24 horas de ser activada.
- Notificar al cliente a respecto de una tarjeta que no transaccionó en los últimos 15 días.

Requisitos de Apple

HST

HST como proveedor de la solución apoya a los Emisores al ofrecer:

- Un SDK completo que cumple con la seguridad, funcionalidades y requerimientos solicitados por Apple.
- Varias opciones de integración con HST.
- Complejidad reducida, ya que el SDK implementa las operaciones criptográficas y el intercambio de certificados con Apple.
- Un conjunto de **APIs de Issuer Server** con las funcionalidades que requiere Apple en sus *Guidelines*.
- Un dashboard de *PowerBI* para que los emisores controlen las métricas de Apple.

Las APIs de Issuer Server hacen posible...

La sugerencia de cual flujo de decisión el emisor debe tomar para cada digitalización (**verde**, **amarillo**, **naranja** o **rojo**).

Establecer controles de digitalizaciones por dispositivos.

(Máximo de 10 dispositivos por una tarjeta).

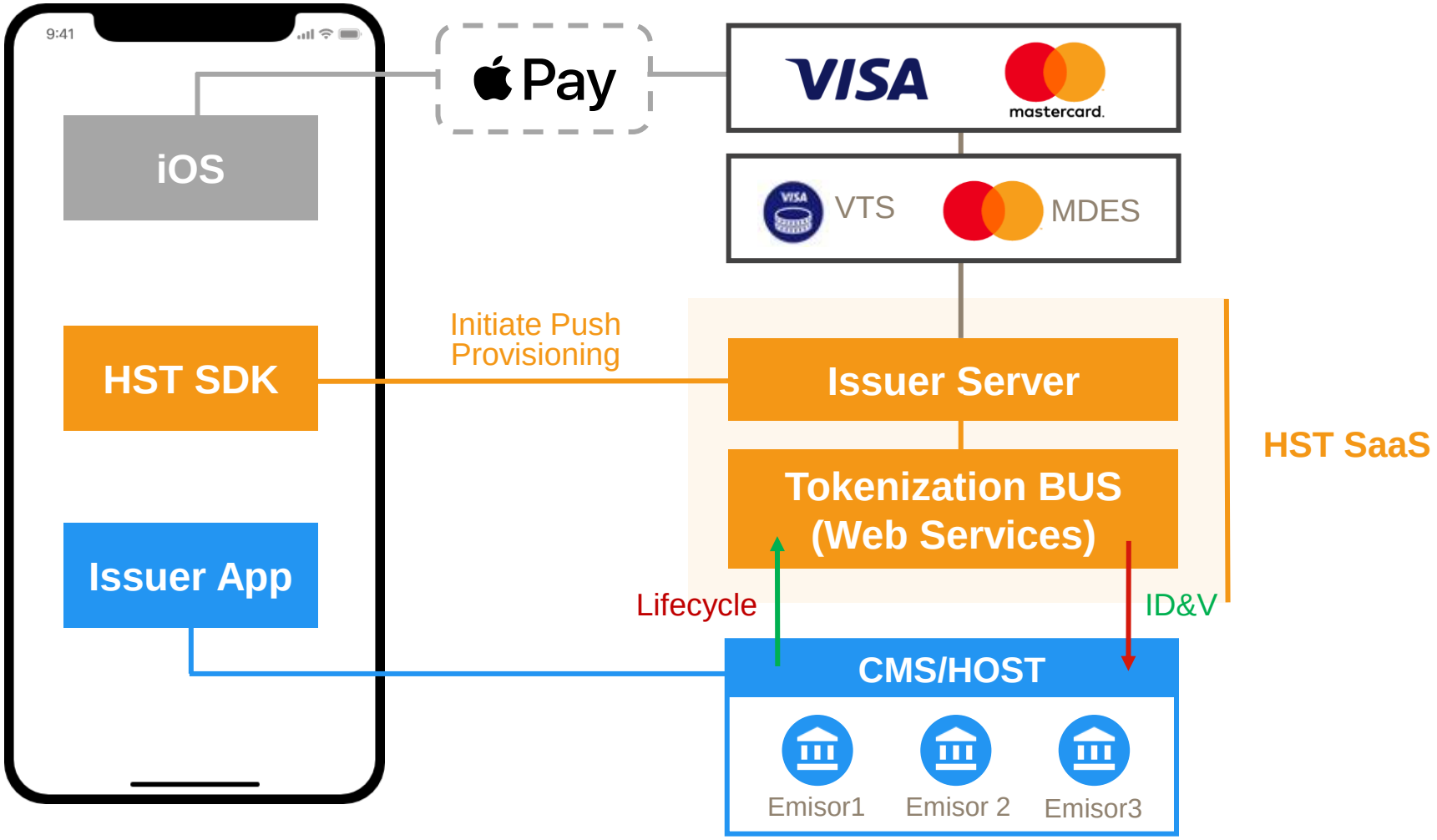
El control de intentos consecutivos de digitalizar una tarjeta con errores de CVV2 o fecha de expiración inválida, para reducir el riesgo de fraude.

Notificaciones para indicar si el token no fue activado en las primeras 24 horas.

HST ofrece estas funcionalidades a traves de APIs para que el Emisor pueda recibir la información necesaria para realizar estos controles.

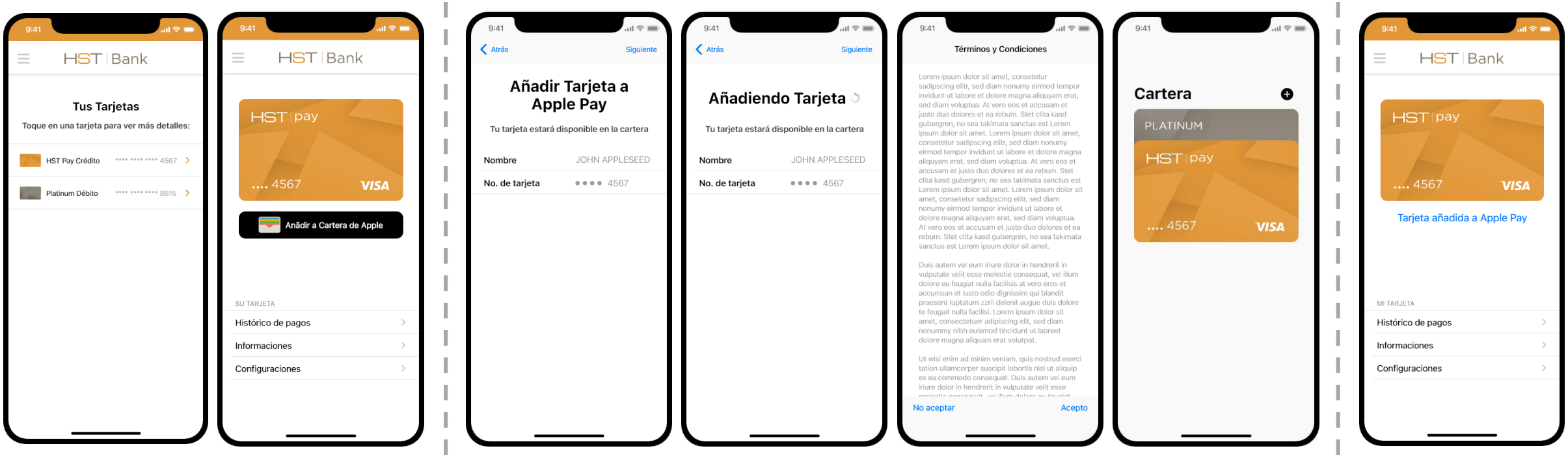
Sin embargo, el Emisor es el responsable por enviar notificaciones e informar el tarjetahabiente sobre estos puntos.

Arquitectura



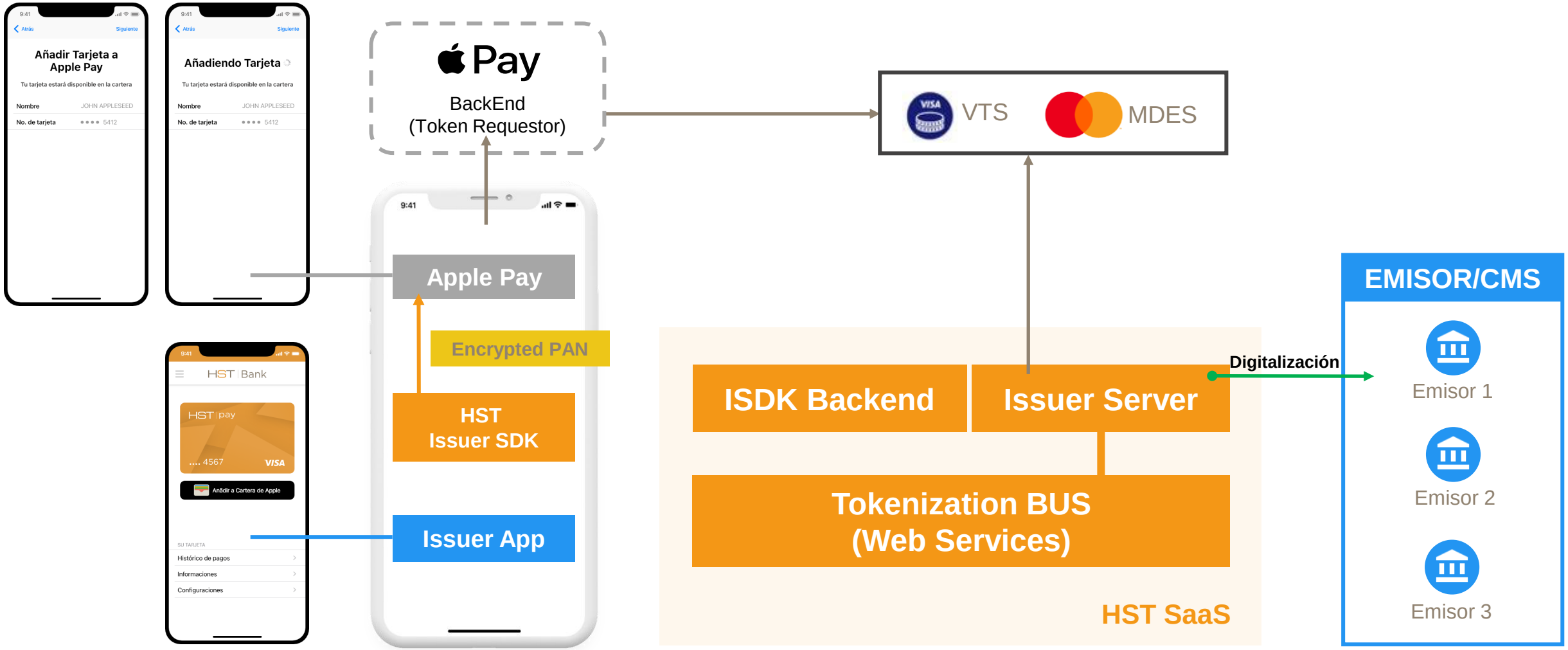
In-App Provisioning

Interfaz Apple

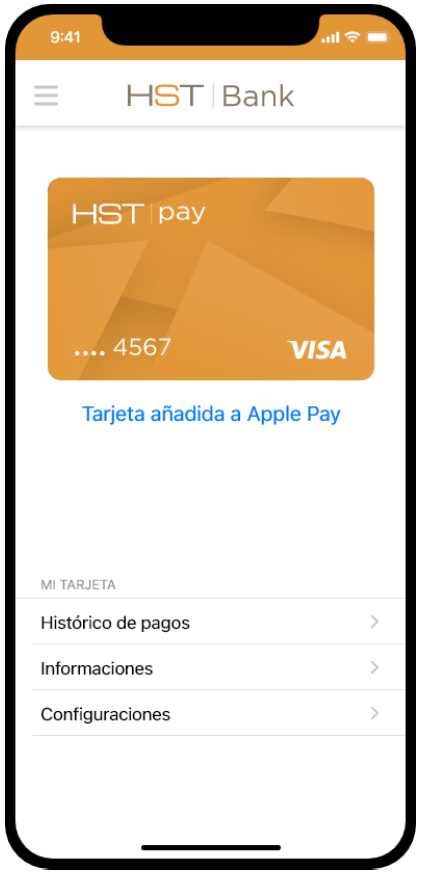


HST provee al Emisor un SDK (componente) para iOS, el cual se integra en el aplicativo móvil del Emisor para habilitar la funcionalidad.

In-App Provisioning



Flujo de Digitalización



1 Tarjetahabiente selecciona la tarjeta a digitalizar

2 Apple Pay recibe el request para digitalizar una tarjeta

Pay
(Token Requestor)

3 Apple solicita la digitalización

4 Vault desencripta el PAN

VTS

MDES

7 Vault provisiona el token

Issuer Server

HST SaaS

5 Emisor recibe el request de **CheckEligibility**

6 Emisor responde: Flujo verde o amarillo

Emisor

8 Token provisionado en Apple Pay

Énfasis a la API CheckEligibility para Apple Pay

tokenRequestorID (TRID) **Check Eligibility Request**
VTS: 40010030273 | MDES: 50110030273

Source
In-App Provisioning: **MOBILE_APP**
Tarjetahabiente ingresó los datos de la tarjeta manualmente (o sacó una foto): **MANUALLY**

walletID
Representa el deviceID (SEID)

riskInformation.RecommendedDecision
“GREEN” o “YELLOW” o “ORANGE” o “RED”

- Si **Orange**, indica que Apple reportó un score de riesgo más grande. Verificaciones de seguridad adicionales deben ser tomadas por el emisor.

recommendedDecision
“GREEN” o “YELLOW” o “RED”

- Si **Red**, estará presente el elemento “**recommendedDecisionReasonCode**” explicando la razón de la recomendación de rechazo/análisis adicional.

returnCode **Check Eligibility Response**

“00” – Tarjeta es elegible para digitalización.
“05” – Tarjeta no es elegible para digitalización;
“85” – Tarjeta es elegible para digitalización y el tarjetahabiente debe ser validado;
“16” – Tarjeta no encontrada, PAN invalido;
“22” – Código de seguridad de la tarjeta (CVV2) inválido;
“23” – Fecha de expiración de la tarjeta inválida;
“27” – Intentos demasiados, sospecha de fraude. Retorno esperado si el valor del elemento "**recommendedDecisionReasonCode**" recibido en el *request* es "**0002**".

Check Eligibility Request


```
{     "requestID": "2",
      "institutionCode": "HST-123",
      "vaultIdentification": "VTS",
      "walletID":
      "042911CB1C1A9002228401698231791 1FC9C04DF0F6F3DA5",
      "tokenRequestorID": "40010030273",
      "tokenRefID": "DNITHE12345123451234500",
      "PANRefID": "V-123451234512345123450 0",
      "encryptedCardInfo": {
            "algorithm": "aes-ccm128",
            "nonce": "a96b3e84232d573c6592ceda",
            "encryptedData":
            "KV1Mgkv40Nt4yggF1Ka7osdIkyMSsVe8K3o9wpQpMRTGeiX
            V2I65fIYgjZY1IGEpj/A7+uAlZ2NazPoWwx5fcQkn",
            "MACLength": 16
      },
      "processID": "1643ef957-622d-4137-abdf-fa605e81e72c",
      "source": “MANUALLY",
      "riskInformation": {
            "recommendedDecision": "YELLOW",
            "deviceScore": "2",
            "accountScore": "2"
      },
      "tokenType": “SE",
      "tokenRequestorName": “Apple Pay",
      "recommendedDecision": “YELLOW ",
}
```


HST
MAKING ELECTRONIC TRANSACTIONS SECURE

Reglas para la API Check Eligibility y Lifecycle Notification

Check Eligibility Request - Elemento **recommendedDecisionReasonCode**
“0001” – Error debido a la misma tarjeta estar digitalizada en demasiados dispositivos.
“0002” – Demasiados intentos consecutivos de digitalización de la tarjeta informando parámetros incorrectos (CVV2 o Fecha de expiración inválida).
“0003” – Recomendación del Token Requestor.

Estos códigos implican que el aprovisionamiento está violando las políticas determinadas por el token Requestor (En este caso, Apple Pay). El emisor debería evaluar/denegar el aprovisionamiento a la marca, **con códigos específicos al contexto:**
Si “0001” es esperado que el emisor responda con returnCode = “05”.
Si “0002” es esperado que el emisor responda con returnCode = “27”
Si “0003” es esperado que el emisor evalúe el objeto riskInformation (datos de riesgo recibidos de Apple) para contestar el returnCode que corresponda (flujo naranja o denegación).

“0002”- Indica que el aprovisionamiento de este token deberá ser denegado por 24h por haber violado los límites permitidos.

Elemento **returnCode** en la respuesta de la API Check Eligibility informa a la marca el flujo de aprovisionamiento indicado por el Emisor:

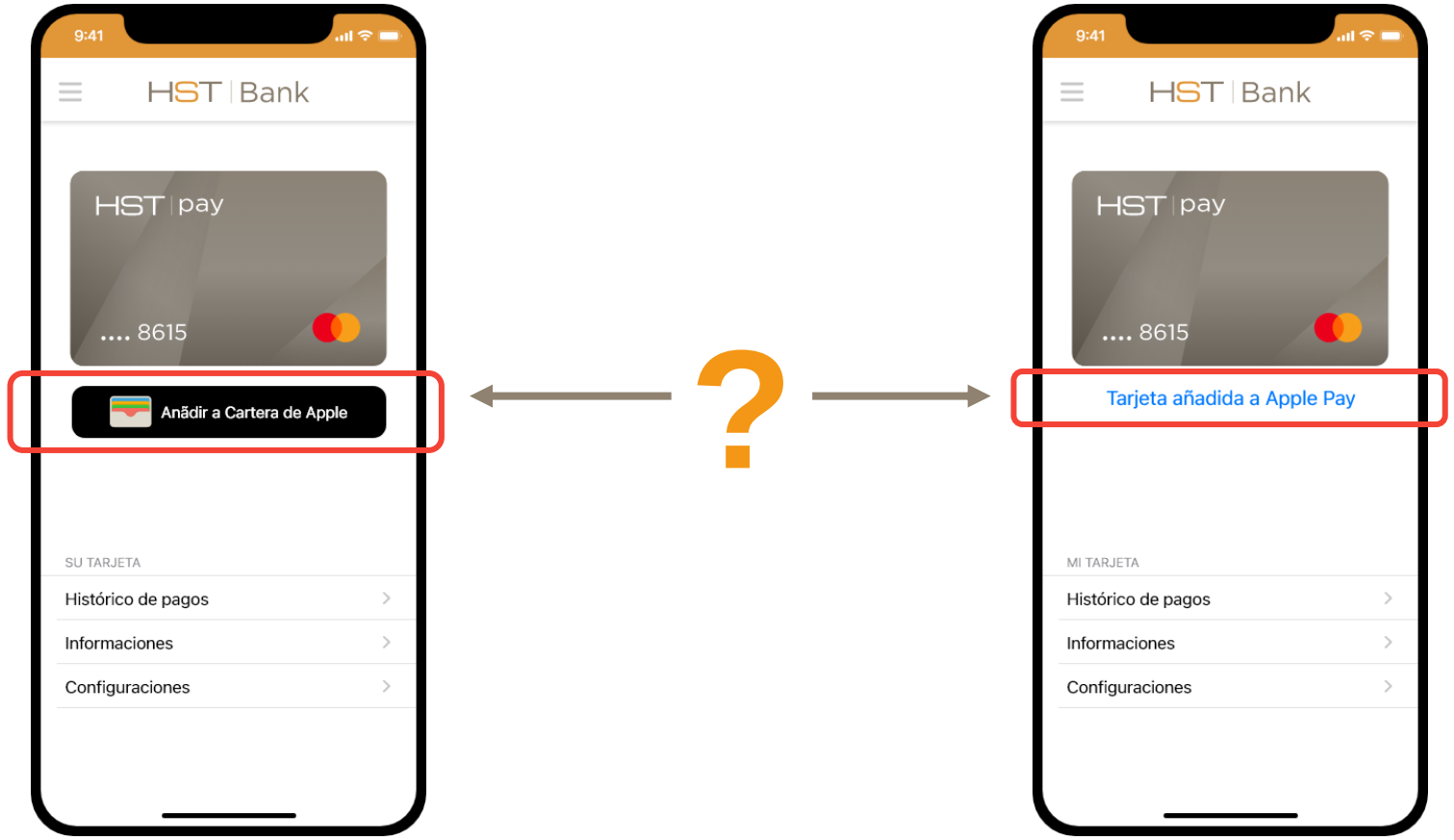
returnCode = “00” -> GREEN (token será aprovisionado con status activo)
returnCode = “85” -> YELLOW (token será aprovisionado con status inactivo – requiere autenticación del TH)
returnCode = “05”,“16”,”22”,”23”,”27” -> RED (rechazo, token no será aprovisionado)

La activación del token puede requerir un proceso de “step-up” (ID&V), donde el emisor indica que antes se debe autenticar el tarjetahabiente.

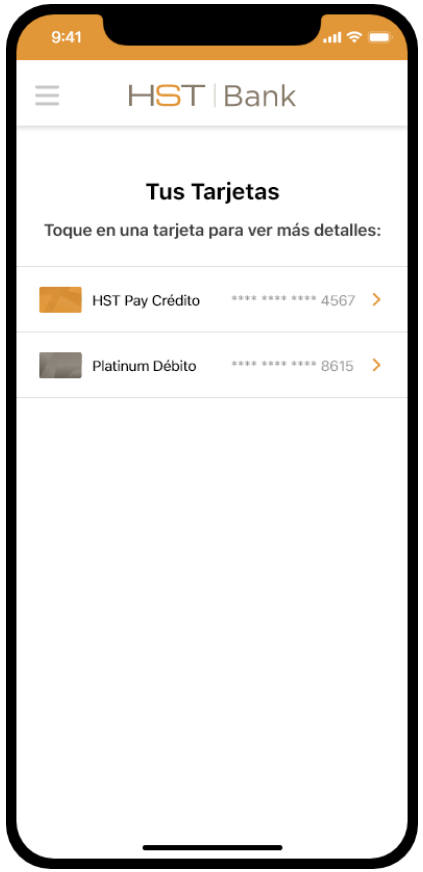
Los tokens aprovisionados **y** no activados serán reportados **a cada 24h vía API Life Cycle Notification (*event* = “**PENDING_ACTIVATION”).

MÉTODOS DEL SDK

Paso 1 Seleccionar la tarjeta



Paso 1 Seleccionar la tarjeta



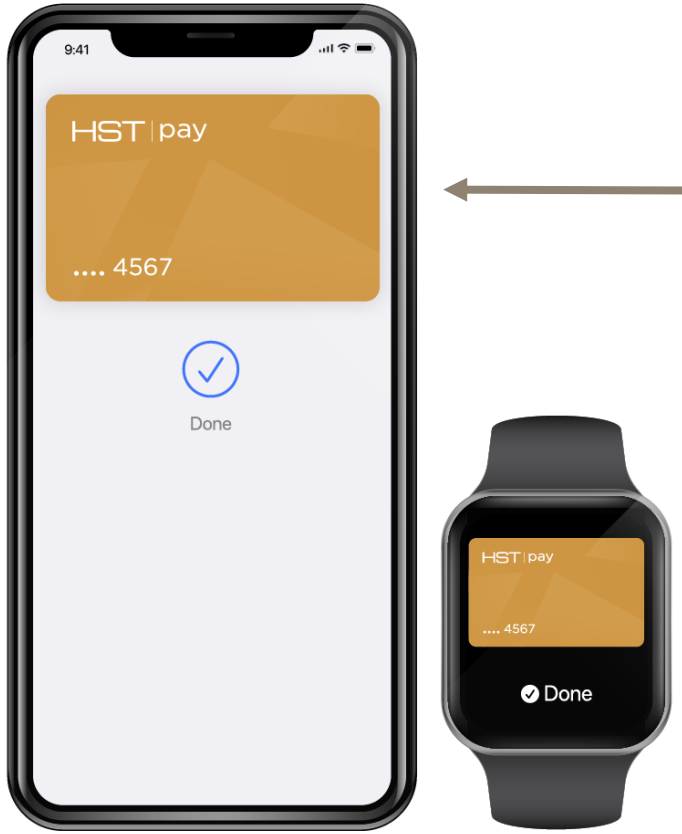
Issuer SDK

Function **getCards**()

Datos retornados para cada tarjeta:

activationState -> ACTIVATED
panLastFour -> 5678
panId -> V-3018253329239943005544
tokenId -> DNITHE302101556961998459
isRemote -> FALSE/TRUE

Paso 1 Seleccionar la tarjeta



isRemote = False

isRemote = True

Requisitos de Apple

Opciones para integración con HST

Para la integración del Emisor a los servicios de HST, ofrecemos 3 opciones diferentes:

Opciones A y B

Son apropiadas para los Emisores que ya están conectados al backend de HST.

En este caso, no necesitan manejar la información de tarjeta en el dispositivo.

Es necesario integrar API's específicas.

Opción C

El Emisor desarrolla un método en el SDK especifico para encriptar los datos de la tarjeta durante el proceso de aprovisionamiento.

De esta forma, no es necesario integrarse al backend de HST o tampoco desarrollar API's.

Paso 2 Preparar los datos (válido solamente para la opción C)

Después de seleccionar cual tarjeta debe ser digitalizada en Apple Pay, el número de la tarjeta debe ser encriptado utilizando JWE (JSON Web Encryption standard).

El resultado es un campo **encryptedPayload**:

El campo encryptedPayload tiene un objeto JWE CardInfo encriptado utilizando los siguientes algoritmos:

{"**alg**":"RSA-OAEP","enc":"A256GCM"}

CardInfo

{
"**pan**": "9999990123456789",
"**expirationDate**": "12/2021“,
"**name**": "Test"
}

En esta opción, el IssuerBackend **no** necesita implementar cualquier API con HST Issuer Server.

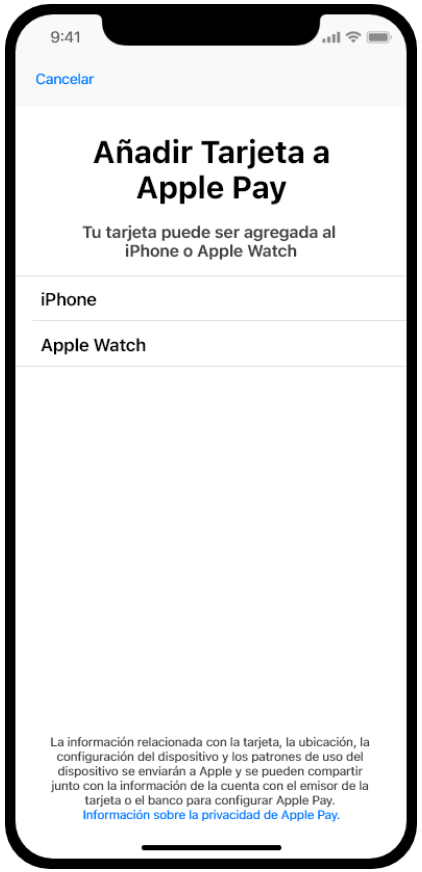
El certificado utilizado para los protocolos JWE es generado por el equipe KMS de HST y enviado al Emisor.

Paso 3 Aprovisionamento (considerando la opción C)

Para digitalizar la tarjeta en Apple Pay, el Emisor debe invocar el método del SDK
executeProvisioningOfEncryptedCard()

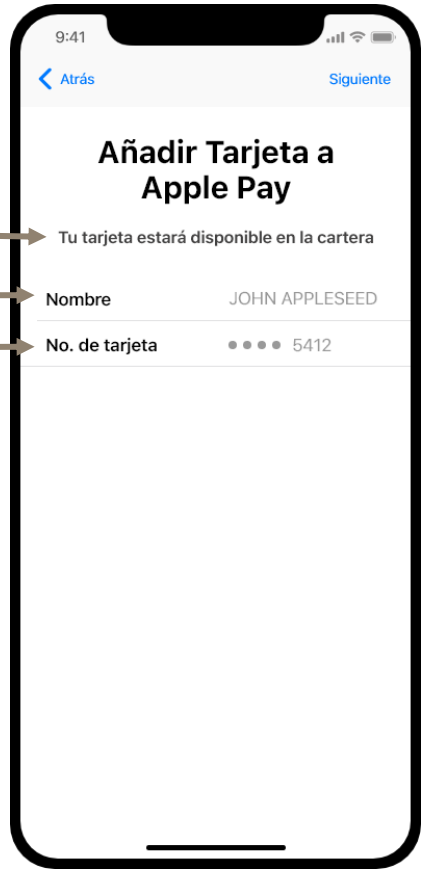
```
public func executeProvisioningOfEncryptedCard (
        parentViewController: ViewController,
        cardholderName: String,
        panSufix: String,
        cardDescr: String,
        panId: String,
        pnp: String,
        encCard: String,
        events: CommEvents?) throws ViewController parentViewController
```

executeProvisioningOfEncryptedCard() (Opción C)

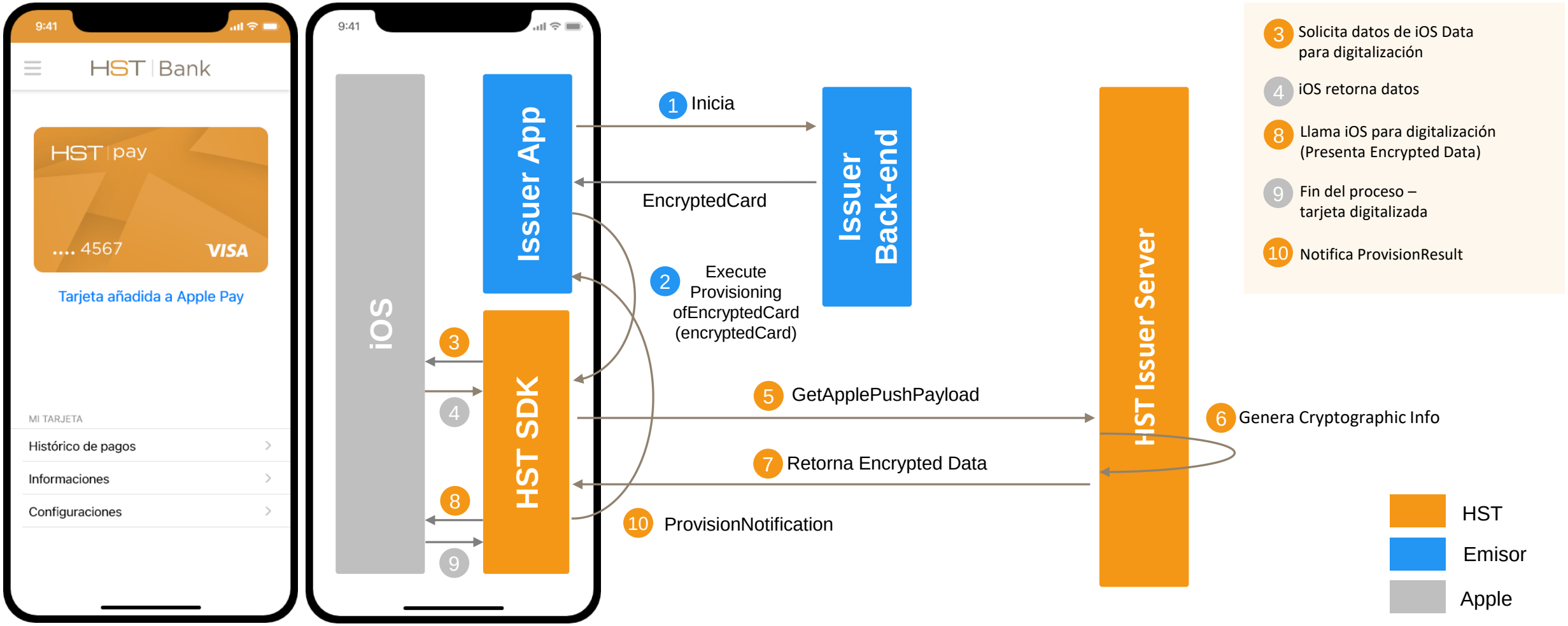


cardDescr: String,
cardholderName: String,
panSufix: String,
panId: String,
pnp: String, (optional)
encCard: String,

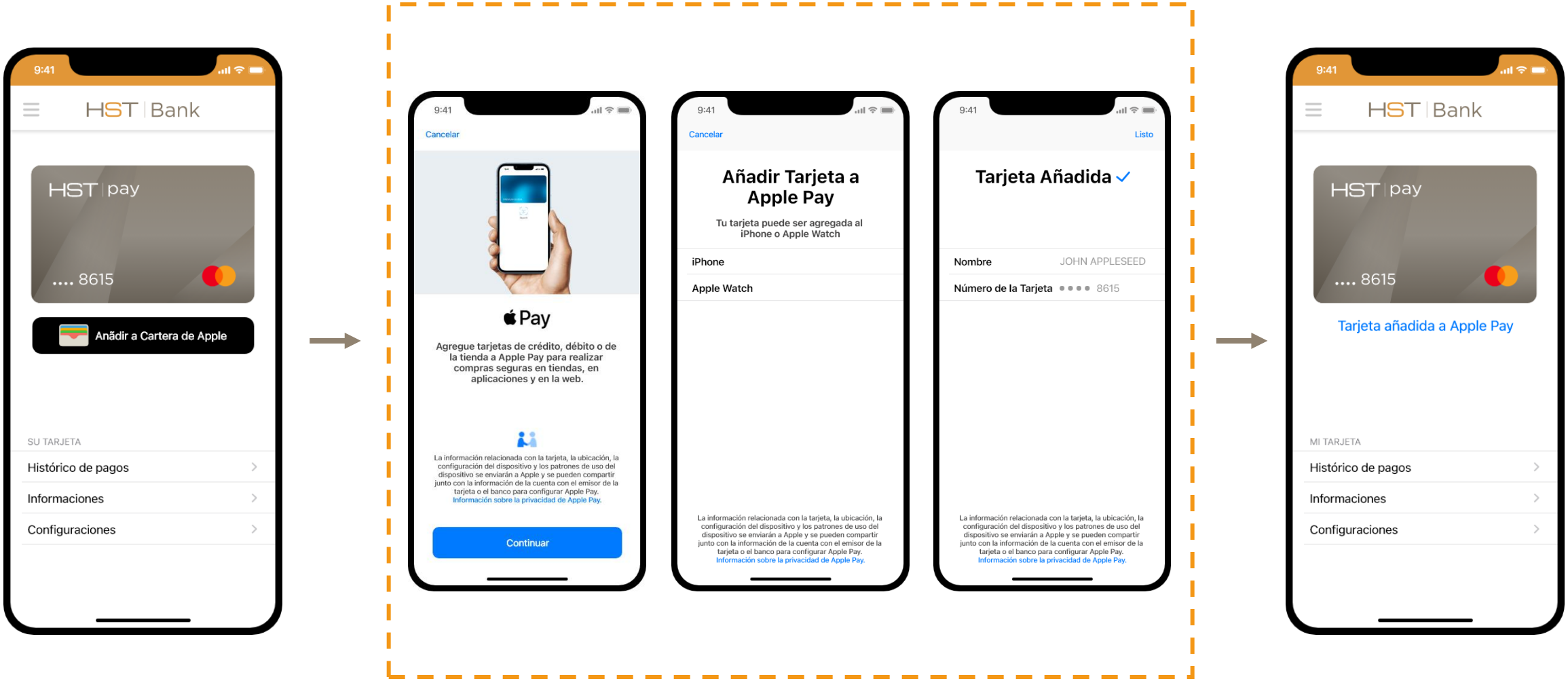
Campo **encryptedPayload** (*JWE Standard*)



In-App Provisioning – Opción C

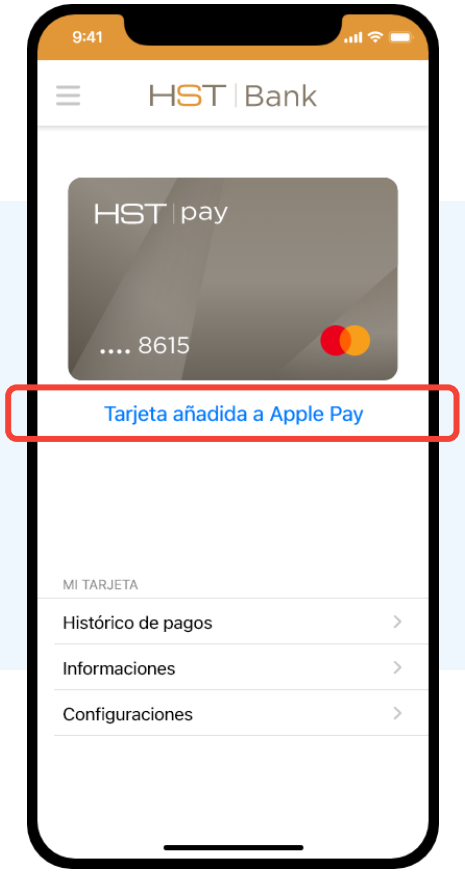


executeProvisioningOfEncryptedCard() (Opción C)



Al digitalizar la tarjeta...

App del Emisor



PROCESO TERMINADO

Tarjeta ha sido digitalizada

Requisitos de Apple

Opciones para integración con HST

Para la integración del Emisor a los servicios de HST, ofrecemos 3 opciones diferentes:

Opciones A y B

Son apropiadas para los Emisores que ya están conectados al backend de HST.

En este caso, no necesitan manejar la información de tarjeta en el dispositivo.

Es necesario integrar API's específicas.

Opción C

El Emisor desarrolla un método en el SDK especifico para encriptar los datos de la tarjeta durante el proceso de aprovisionamiento.

De esta forma, no es necesario integrarse al backend de HST o tampoco desarrollar API's.

Paso 2 Opción A

Opción A: El Emisor solicita al backend de HST un push receipt ID del proceso, que será usado posteriormente para consultar el emisor sobre los datos de la tarjeta durante el proceso de *in-app push provisioning*.

IssuerBackend necesita desarrollar 2 APIs:

1. ***GetPushReceipt()*** > ***PushReceiptID***

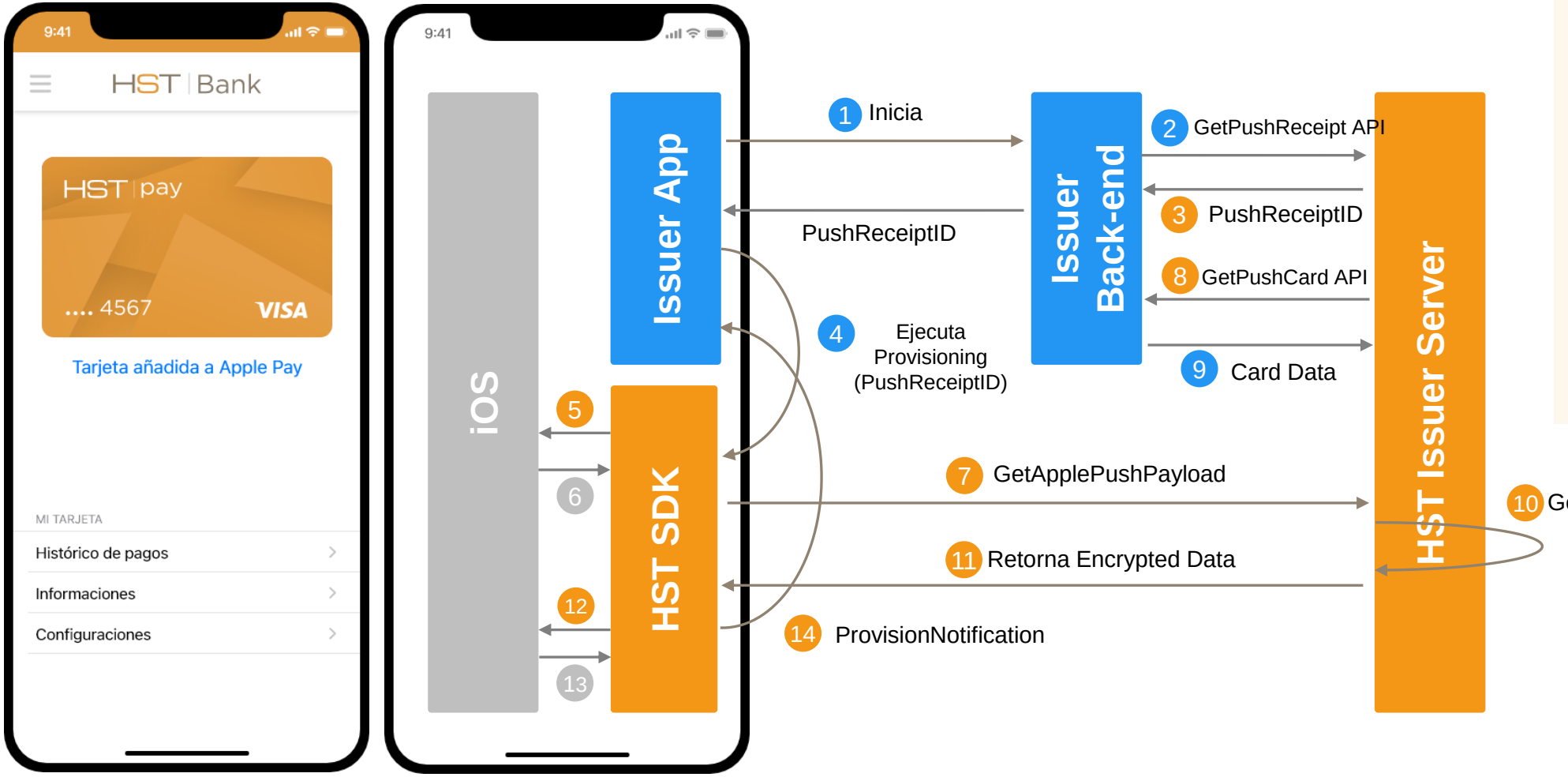
2. ***GetPushCard()*** > ***CardInfo***

1. En esta opción de push provisioning, el IssuerApp solicita a HST, vía IssuerBackend, un recibo temporario para identificación de este flujo de push: **pushReceiptID**.

2. Este código es utilizado por el **IssuerBackend** para vincularlo a los datos de la tarjeta a ser aprovisionada en la Apple Wallet. El IssuerBackend mantiene el *mapping* de referencia a los datos de la tarjeta.

3. El SDK ejecuta el push provisioning en la Wallet iOS vía método **executeProvisioning(pushReceiptID)**, y entonces en este proceso serán informados los datos de la tarjeta a Apple, es decir, el Issuer Backend será consultado utilizándose el código generado en el paso **1**.

A In-App Provisioning



5 Solicita datos de iOS Data para digitalización

6 iOS retorna datos

8 Presenta Receipt ID al emisor & retorna datos de tarjeta

12 Llama iOS para digitalización (Presenta Encrypted Data)

13 Fin del proceso – tarjeta digitalizada

14 Notifica ProvisionResult

Paso 2 Opción B

Opción B: El Emisor informa los datos de la tarjeta al backend de HST y recibe un push receipt ID que será utilizado posteriormente para asociarlo a las informaciones de la tarjeta durante el proceso de *in-app push provisioning*.

IssuerBackend necesita desarrollar 1 API:

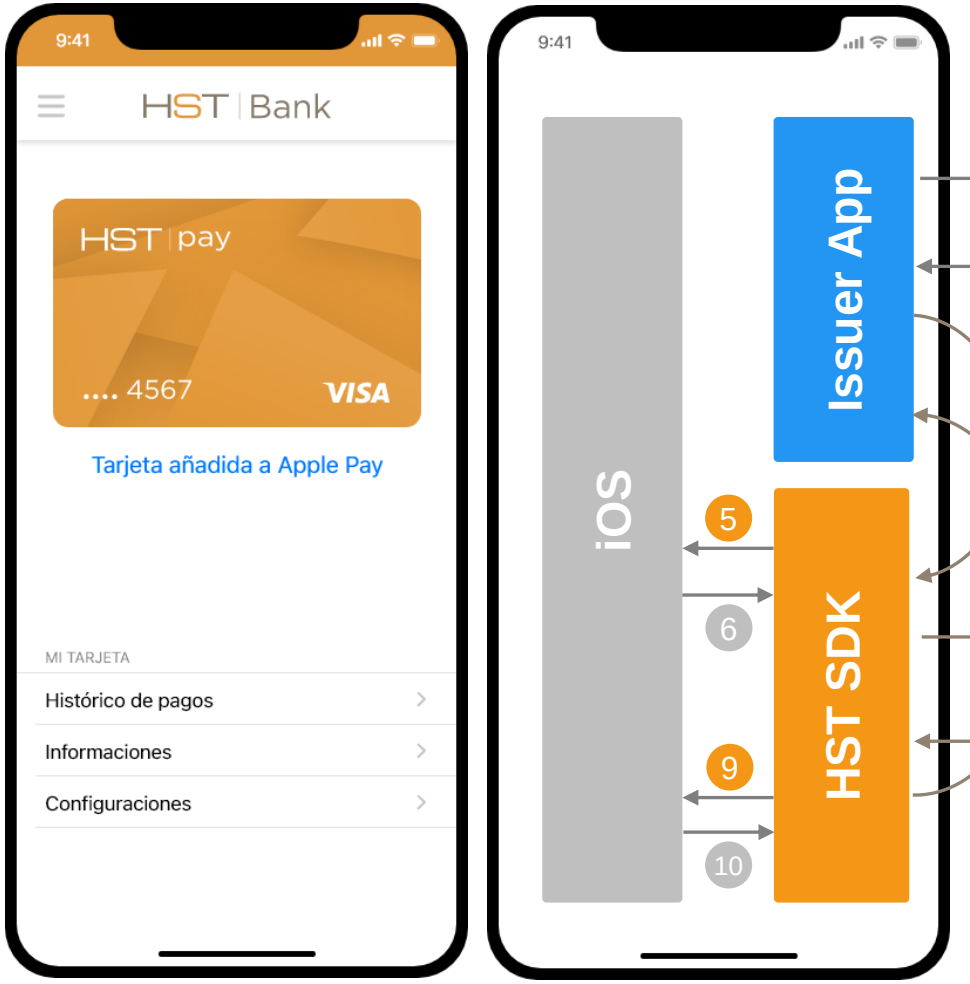
1. ***GetPushReceipt()*** > ***CardInfo***

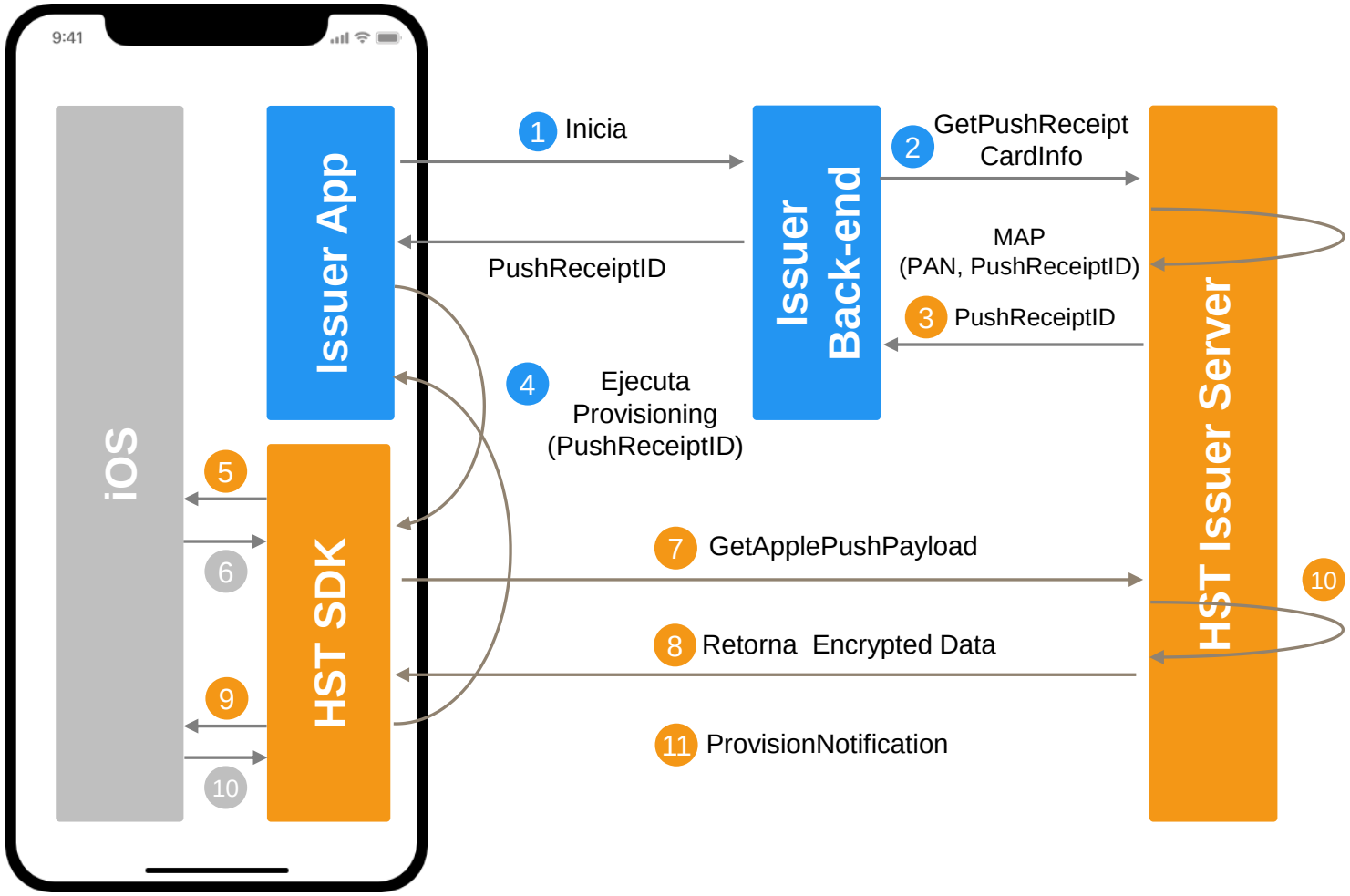
1. En esta opción de push provisioning el Issuer App informa los datos de la tarjeta al HST vía IssuerBackend, donde se genera un recibo temporario para identificación de este proceso de push: **pushReceiptID**.

2. Este código es utilizado por **HST** para asociarlo a los datos de la tarjeta a ser aprovisionada en Apple Wallet.
En esta opción es HST que mantiene, de forma temporaria, el mapping de la referencia a los datos de la tarjeta.

3. El SDK realiza el aprovisionamiento en la Wallet iOS vía método **executeProvisioning(pushReceiptID)**, y entonces en este proceso serán informados los datos de la tarjeta a Apple, es decir, HST va a utilizar el código generado no paso **1** para recuperar los datos de la tarjeta.

B In-App Provisioning





5 Solicita datos de iOS Data para digitalización

6 iOS retorna datos

9 Llama iOS para digitalización (Presenta Encrypted Data)

10 Fin del proceso – tarjeta digitalizada

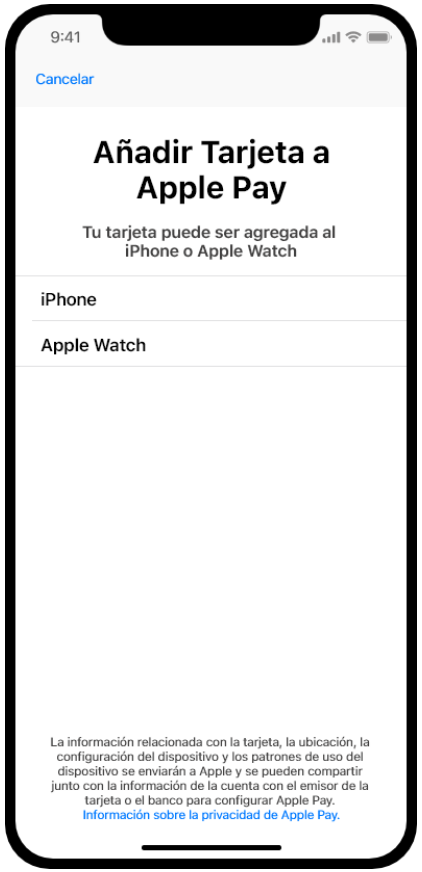
11 Notifica ProvisionResult

HST

Emisor

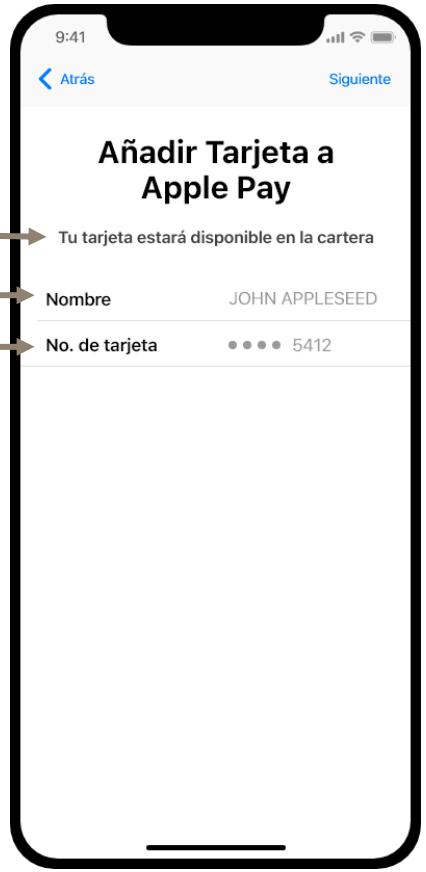
Apple

executeProvisioning() (Opciones A y B)



cardDescr: String,
cardholderName: String,
panSufix: String,
panId: String,
pnp: String, (optional)
pushRecId: String,

Campo **recibido desde el backend de HST**



Paso 2 Comparativo de las opciones de integración

Las opciones A, B y C se difieren en el modo como los datos de tarjeta son manipulados durante el flujo de aprovisionamiento para Apple Wallet:

Opciones **A** y **B**

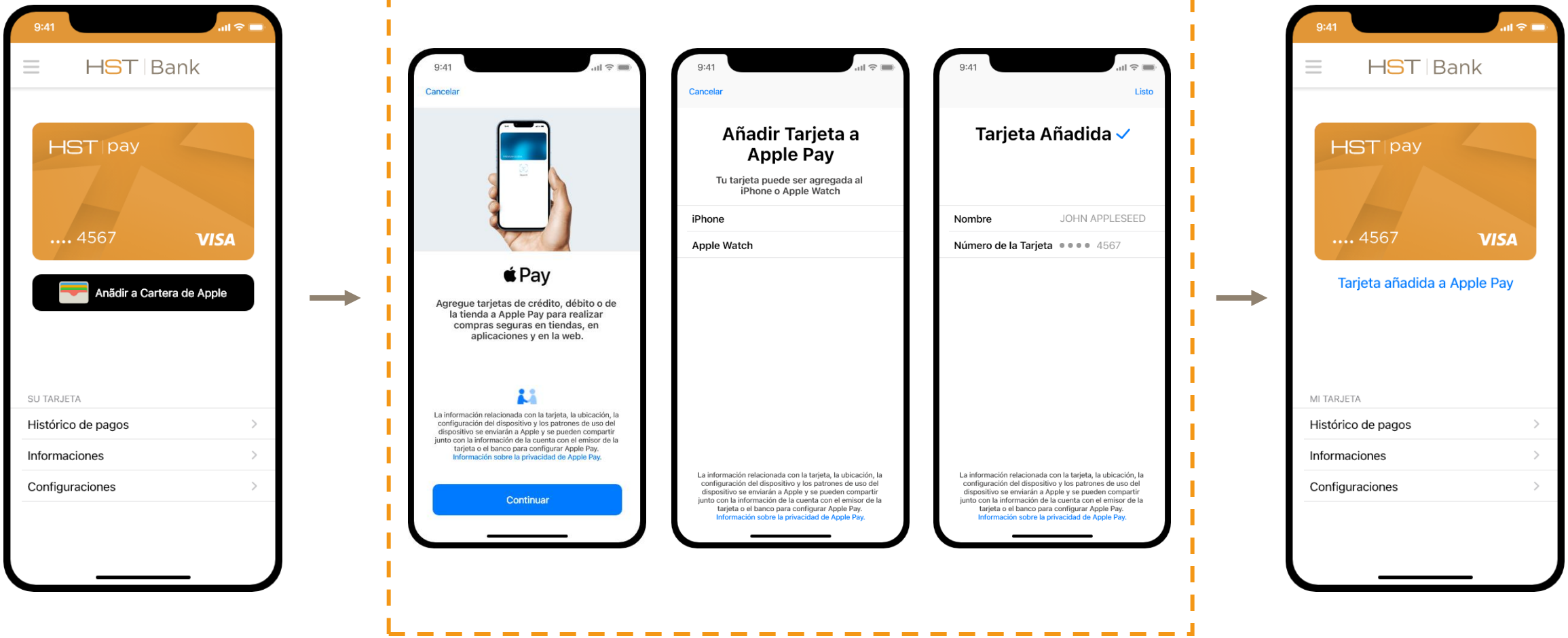
Son recomendadas a Emisores que ya estén conectados con el backend de HST.
Tienen la ventaja de no necesitar manejar datos de la tarjeta, mismo que cifrados, en el dispositivo.
Por otro lado, el Emisor necesita implementar APIs en su backend.

Opción **C**

Es más adecuada para Emisores que deseen minimizar las interacciones con el backend de HST.

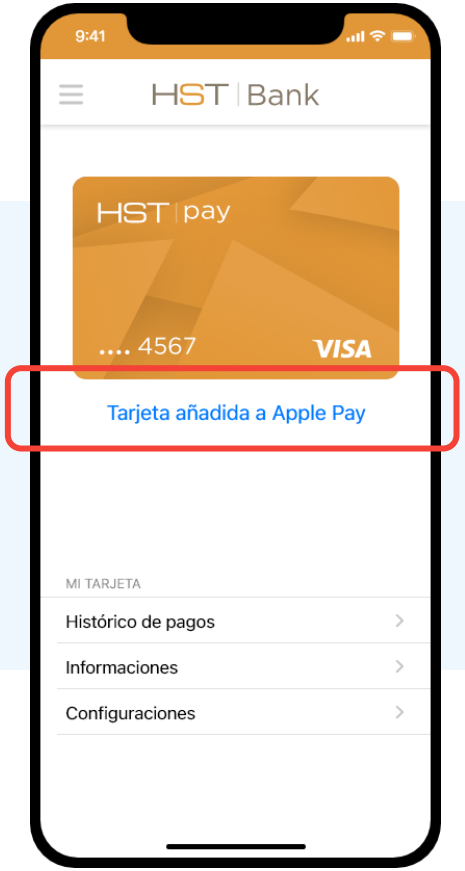
Independente de la opción utilizada, el Emisor nunca debe manejar los datos de sus tarjetas en claro en su banking App.
Todas las operaciones criptográficas bajo la data de las tarjetas deben ser ejecutadas en el backend del Emisor debido a temas de seguridad.

executeProvisioning() (Opciones A y B)



Al digitalizar la tarjeta...

App del emisor



PROCESO TERMINADO

Tarjeta ha sido digitalizada

Resumen de las especificidades de Apple

Check Eligibility **Flujo Regular**

If
source = “MANNUALLY”
recommendedDecision = “YELLOW”
Then
returnCode = “**85**”

If
source = “MOBILE_APP”
recommendedDecision = “GREEN”
Then
returnCode = “**00**”

Check Eligibility **Ressubmission (Visa Only)**

If
riskInformationResubmission = TRUE
Then
The TR has reported RiskInfo, the issuer must inspect the risk Data:

```
{
    “recommendedDecision”: “”,            ← From HST
    “riskInformation” : {
        recommendedDecision: “”,          ← From TR (Apple Pay)
        deviceScore: “[1-5]”,
        accountScore: “[1-5]”
    }
}
```

Check Eligibility **Flujo Naranja**

If
RiskInfo.RecommendedDecision = “ORANGE”
Then
returnCode = “85”
authenticationMethods = {
…
“type” : “customer_service”,
“maskedInfo“ : “1-800-555-555“,
…
}

- Remover las opciones de envío de OTP de la lista de métodos disponibles (e-mail/SMS).
- Mantener opción de call center, pero dirigido a un centro de atención a fraudes.

Check Eligibility **(from slide 19)** **Apple Pay**

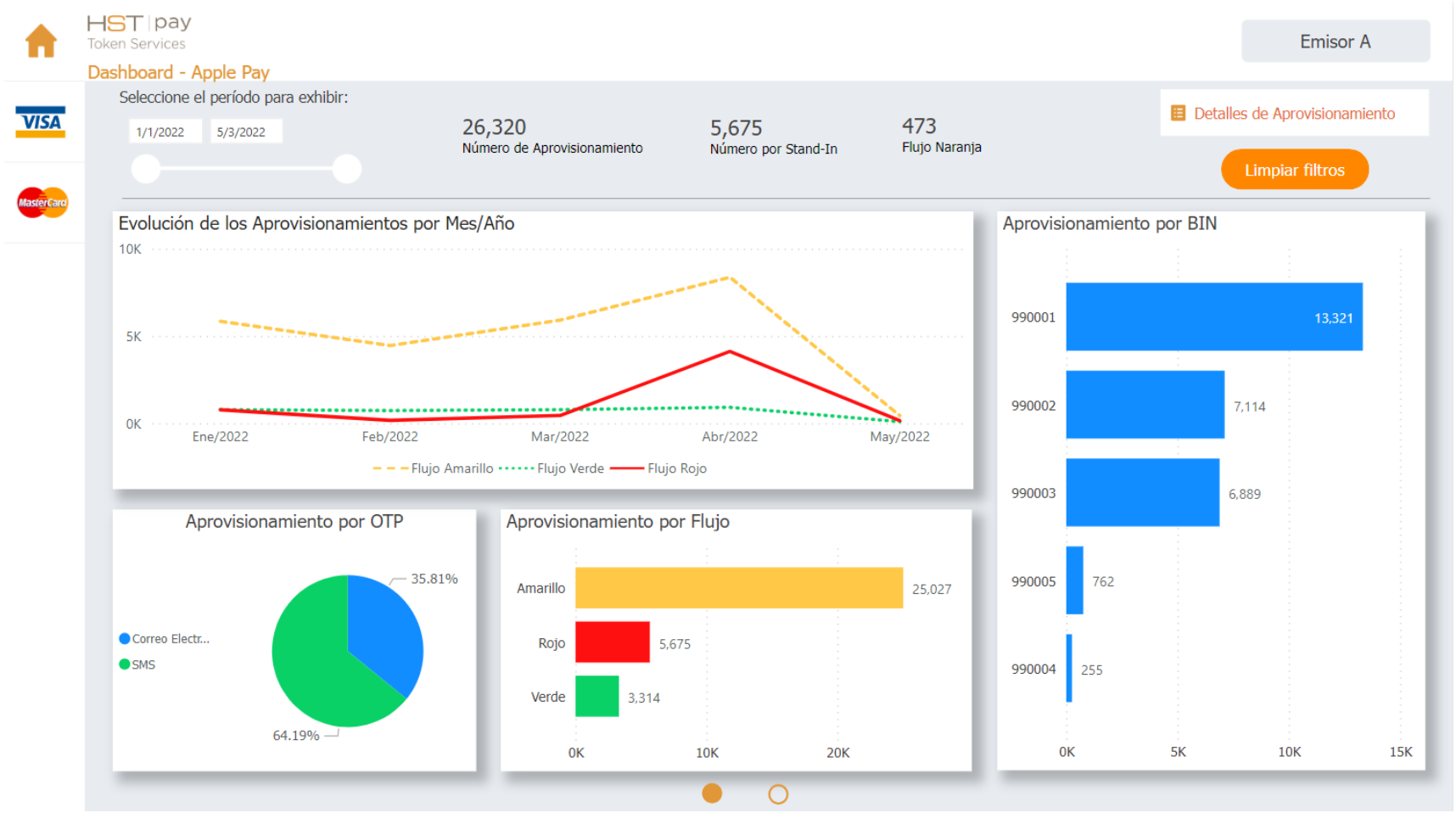
VTS tokenRequestorID = “40010030273”

MDES tokenRequestorID = “50110030273”

walletID = DeviceID (SEID)

Tokens provisioned by **YELLOW** flow that don’t perform ID&V will be notified every 24h, by HST Issuer Server Life Cycle Notification API, until the token activation.

Dashboard Apple Pay



Dashboard Apple Pay

HST pay
Token Services

Emisor A

Dashboard - Apple Pay

Seleccione el período para exhibir: 1/1/2022 5/3/2022

lifeCycleTraceID: All | BIN: All | panSource: All | Dispositivo: All | Flujo: All

Emisor	ID Audit	lifeCycleTraceID	BIN	panSource	Dispositivo	actionCode	Flujo	Fecha
Emisor A	176392889	302001000359576	990001	ON_FILE	TABLET	85	Amarillo	1/1/2022 12:01:21 AM
Emisor A	176393180	302001002304705	990002	ON_FILE	MOBILE_PHONE	85	Amarillo	1/1/2022 12:03:59 AM
Emisor A	176393740	382001005293284	990001	ON_FILE	MOBILE_PHONE	85	Amarillo	1/1/2022 12:08:57 AM
Emisor A	176394849	462001011266910	990001	ON_FILE	WATCH	85	Amarillo	1/1/2022 12:19:22 AM
Emisor A	176394886	462001011708069	990003	ON_FILE	MOBILE_PHONE	85	Amarillo	1/1/2022 12:19:47 AM
Emisor A	176396045	382001018760535	990001	ON_FILE	MOBILE_PHONE	85	Amarillo	1/1/2022 12:31:30 AM
Emisor A	176396216	462001020075376	990001	MOBILE_BANKING_APP	MOBILE_PHONE	0	Verde	1/1/2022 12:33:48 AM
Emisor A	176396869	462001024007739	990002	ON_FILE	MOBILE_PHONE	5	Rojo	1/1/2022 12:40:17 AM
Emisor A	176398552	582001034618880	990001	KEY_ENTERED	MOBILE_PHONE	85	Amarillo	1/1/2022 12:57:54 AM
Emisor A	176398865	462001036503365	990001	ON_FILE	WATCH	85	Amarillo	1/1/2022 1:01:21 AM
Emisor A	176399399	462001040027035	990001	ON_FILE	MOBILE_PHONE	85	Amarillo	1/1/2022 1:07:11 AM
Emisor A	176399700	582001042265015	990001	KEY_ENTERED	MOBILE_PHONE	85	Amarillo	1/1/2022 1:10:38 AM
Emisor A	176399823	462001042977121	990001	ON_FILE	WATCH	85	Amarillo	1/1/2022 1:12:22 AM
Emisor A	176400874	462001050729874	990002	KEY_ENTERED	WATCH	85	Amarillo	1/1/2022 1:24:41 AM
Emisor A	176401284	582001054953300	990003	ON_FILE	MOBILE_PHONE	85	Amarillo	1/1/2022 1:31:47 AM

SDK: Flujo de inicialización

4.2.1. Initialize

Description:	This class constructor is responsible for initializing the SDK. It must be called before any other method. Due to security reasons, it is extremely recommended to call this class constructor during application startup.
Prototype:	public init (institutionCode: String)
Input Param:	*String institutionCode* The financial institution code provided during onboarding on HST Tokenization Cloud. Contact your technical support to get it.

4.2.3. Is Available

Description:	Checks if the device has the capability to digitize cards.
Prototype:	public func isAvailable ()-> Bool
Input Param:	None.
Return:	True if the device is available for card digitization, False otherwise.

4.2.4. Is Available for Card

Description:	Checks if the specific card is available or not for digitization.
Prototype:	public func isAvailableForCard (panRefId: String) -> Bool
Input Param:	*String panRefId* The PAN Reference ID returned on the first digitization of the card on the device.
Return:	True if the card is available for digitization, False otherwise.

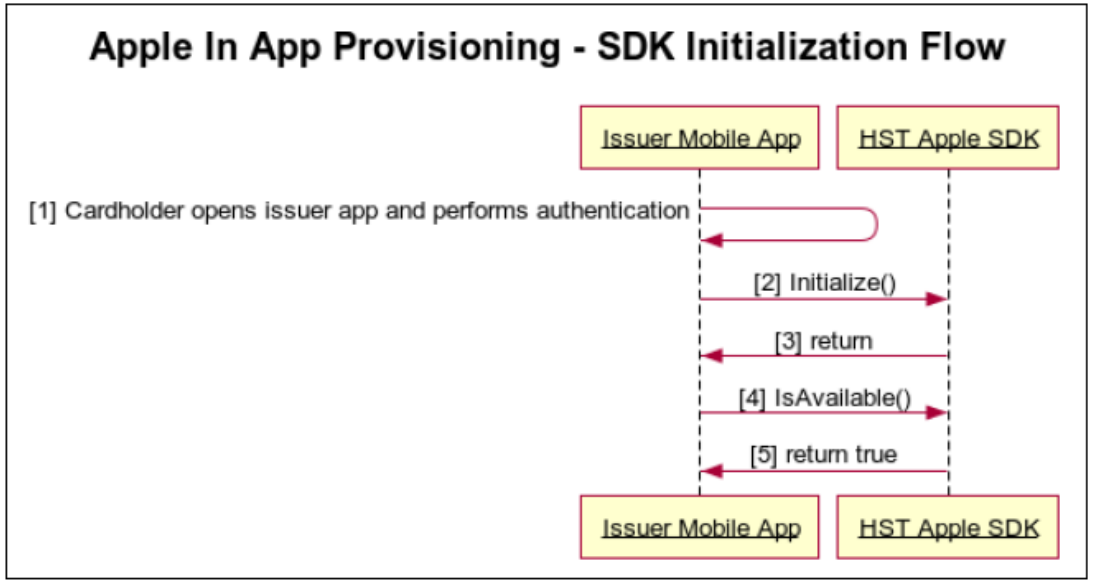


Figure 1- Initialization Flow

Otros flujos y más detalles en la documentación técnica del producto

4.1 Casos de uso y diagramas de secuencia - *Apple In App Provisioning* – Flujo de selección de la tarjeta

4.1.2. Adding a new card in Apple Pay

Escenario 1 – Tarjeta todavía no aprovisionada en Apple Pay

4.1.3. Adding a card in another device

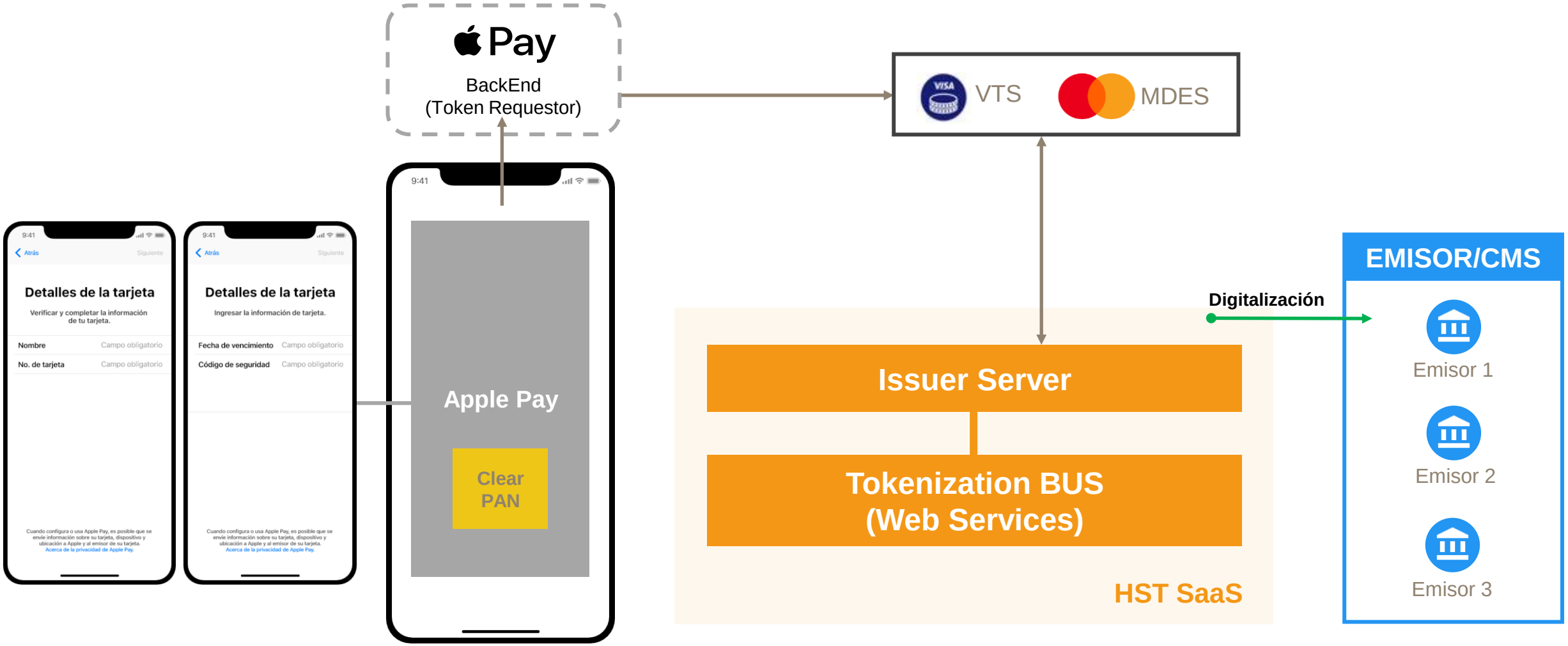
Escenario 2 - Tarjeta ya aprovisionada en un dispositivo, pero que todavía puede ser “empujada” para otro dispositivo

4.1.4. Card already provisioned in all available devices

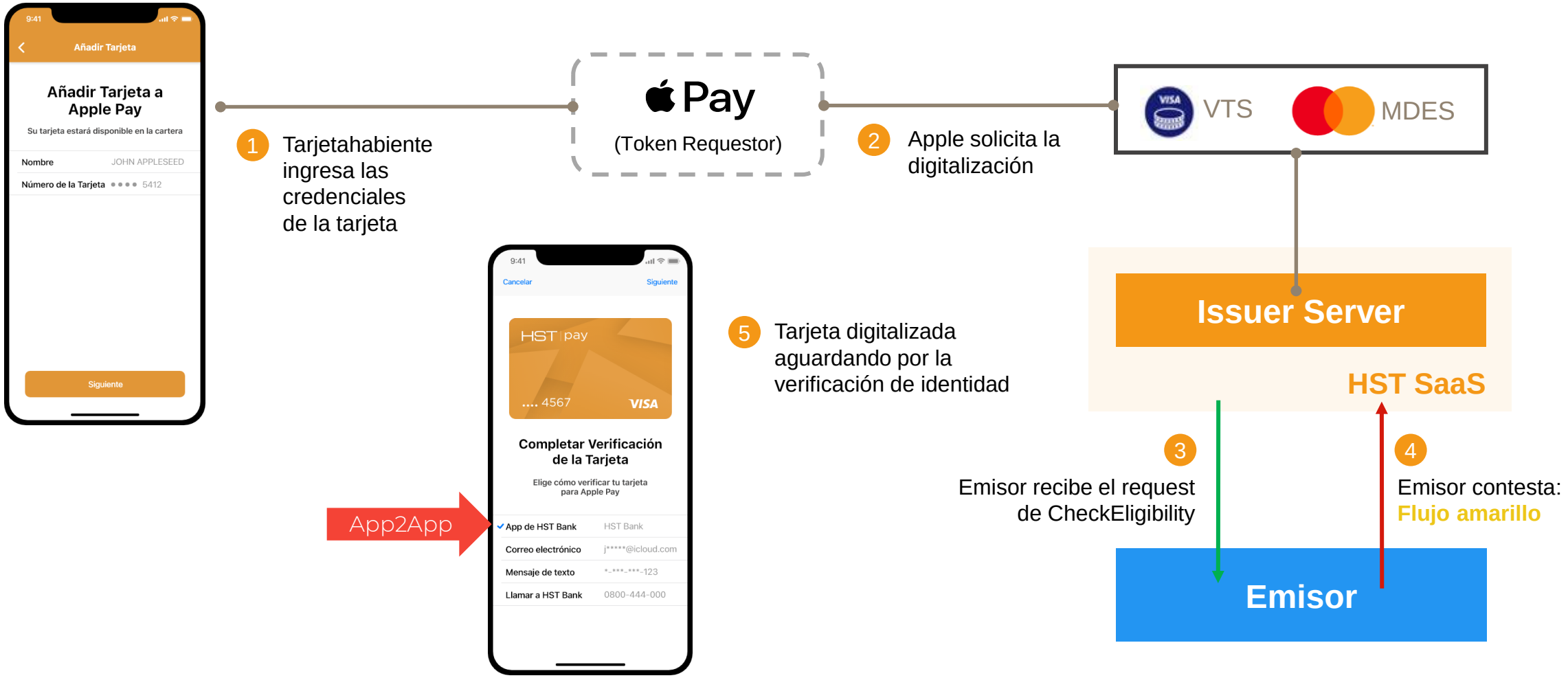
Escenario 3 – Tarjeta ya aprovisionada en todos los dispositivos disponibles

2 In-App Verification

In-App Verification



Flujo de Digitalización



Flujo Amarillo

Métodos de Autenticación

Pasos 4 y 5

App to App
(ID del app)

Objeto authenticationMethods **CheckEligibility Response**
Lista de métodos de autenticación para una tarjeta específica, si la autenticación es necesaria. Los valores posibles para implementar son *OTP*, *Call Center* y *App-to-App,* y sus detalles son descritos en la documentación técnica en el elemento AuthenticationMethod.

AuthenticationMethod Object

```
{
    "type": "bank_app",
    "maskedInfo":"My Mobile Banking App",
    "sourceAddress": "com.DemoBank.DemoApp",
    "platform" : "IOS" o "ANDROID" o "WINDOWS" o "WEB"
}
```

Apple Adam ID

Flujo Amarillo

Métodos de Autenticación

Pasos 4 y 5

Call center

Objeto authenticationMethods **CheckEligibility Response**

Lista de métodos de autenticación para una tarjeta específica, si la autenticación es necesaria. Los valores posibles para implementar son *OTP*, *Call Center* y *App-to-App,* y sus detalles son descritos en la documentación técnica en el elemento AuthenticationMethod

AuthenticationMethod Object

```
{
        "type": "customer_service",
        "maskedInfo":"1-800-555-555",

}
```

Flujo Amarillo

Métodos de Autenticación

Pasos 4 y 5

OTP
(E-mail / SMS)

Objeto authenticationMethods **CheckEligibility Response**

Lista de métodos de autenticación para una tarjeta específica, si la autenticación es necesaria. Los valores posibles para implementar son *OTP*, *Call Center* y *App-to-App,* y sus detalles son descritos en la documentación técnica en el elemento AuthenticationMethod

AuthenticationMethod Object

```
{
    "type": "email" o "cell_phone",
    "maskedInfo":"ip****@gmail.com" o "********12" ,

}
```

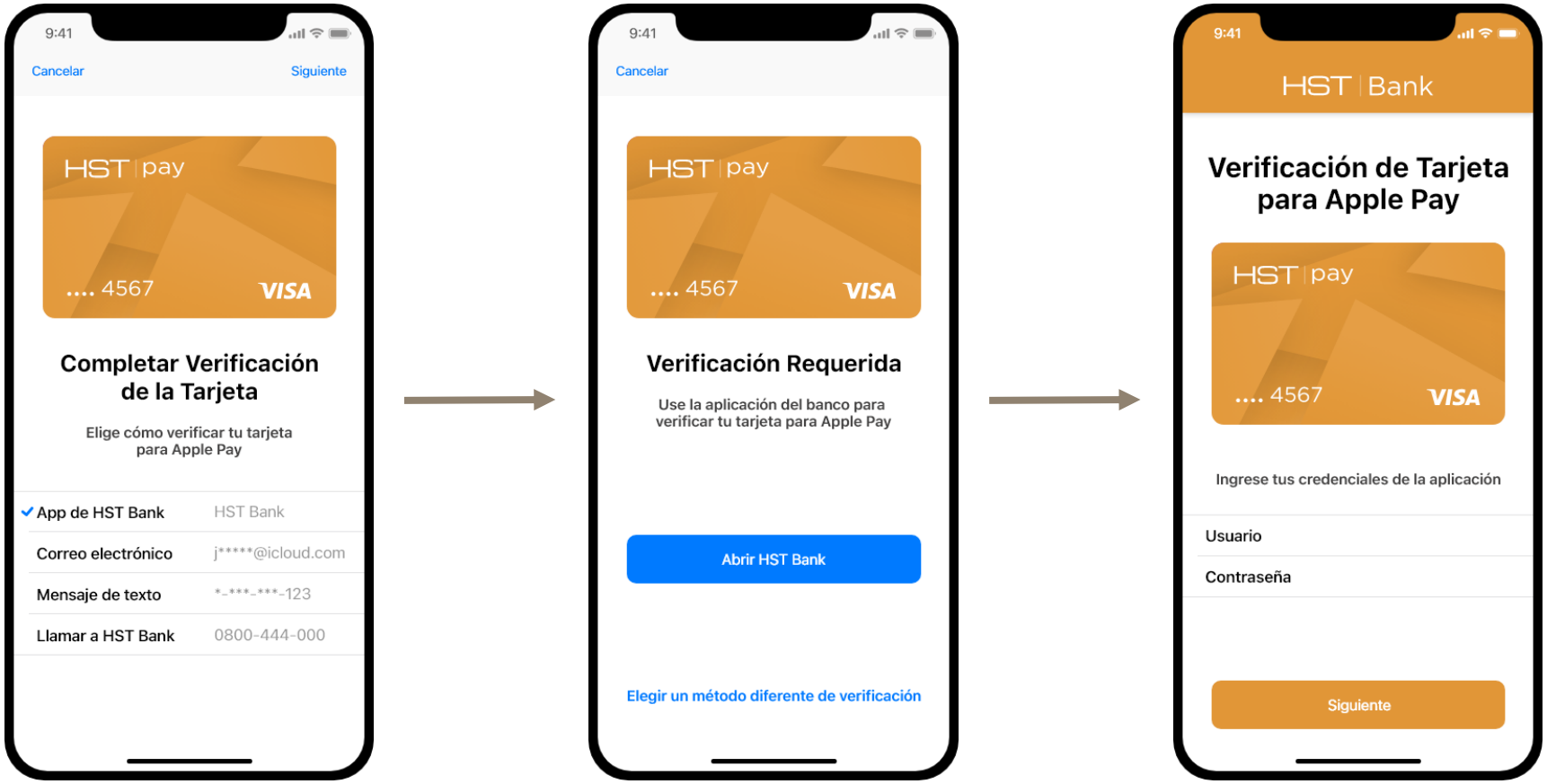
Paso 5 App to App launch Intent()

La Issuer app recibe el *payload* en un formato de *encoded* string.
La string es codificada en base64URL y encapsula el *payload* JSON.

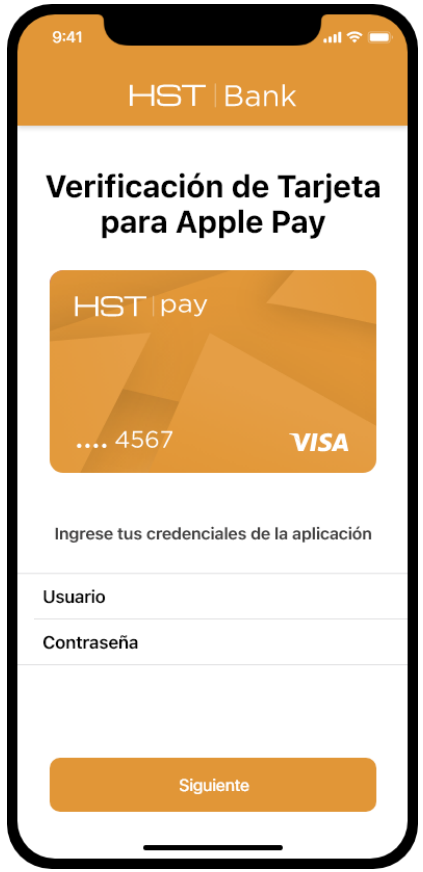
Este ejemplo indica la representación del *payload*:


```
{
    "panReferenceID" : "V-3815023863409817870482",
    "tokenRequestorID" : "42301999123",
    "tokenReferenceID" : "DNITHE381502386342002358",
    "panLast4" : "1234",
    "deviceID" : "DEiOiJBMjU2R_0NNS1-ciLCJiI",
    "walletAccountID" : "AiOiJBMjU-2_R0NNS1ciLCJiI6"
}
```

Después del “launch” de la app - Verificar la tarjeta



¿Cómo identificar cual tarjeta activar?



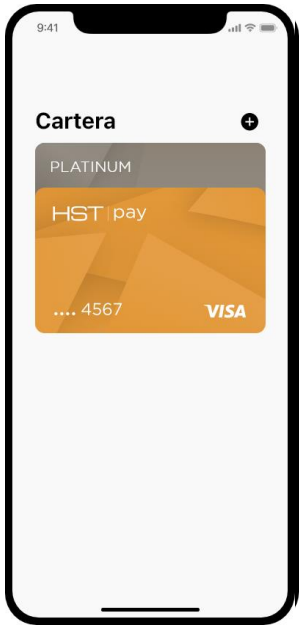
Issuer SDK

Método **getCards**()

Datos retornados para cada tarjeta:

CardStatus	>	REQUIRES_ACTIVATION
panLastFour	>	4567
panId	>	V-3018253329239943005544
tokenId	>	DNITHE3021015569619984 59

Flujo de Activación de la Tarjeta



4 Tarjeta activada en Apple Pay

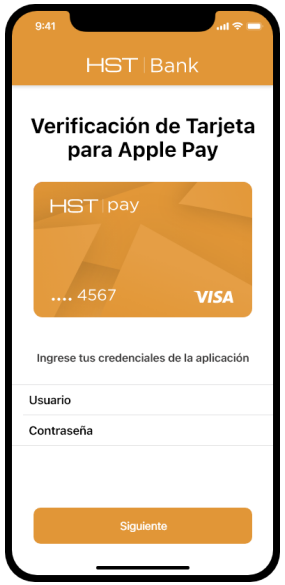
3 Token Requestor es notificado de la activación del token

Pay
(Token Requestor)

VTS
MDES

Issuer Server

HST SaaS



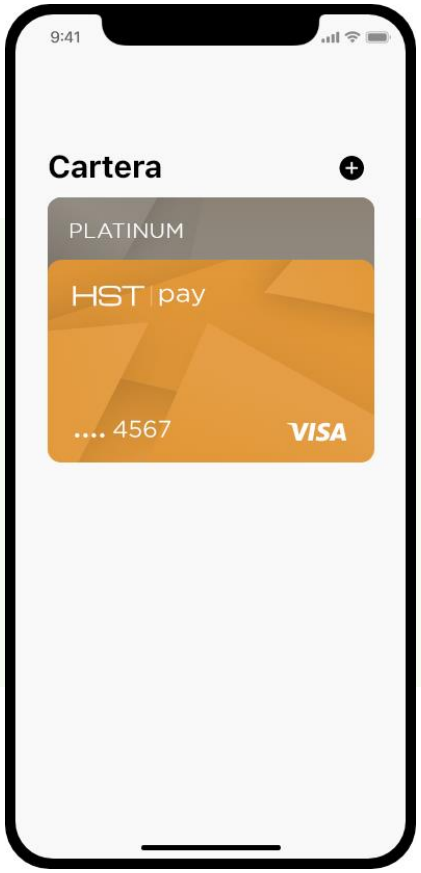
1 Emisor autentica el Tarjetahabiente

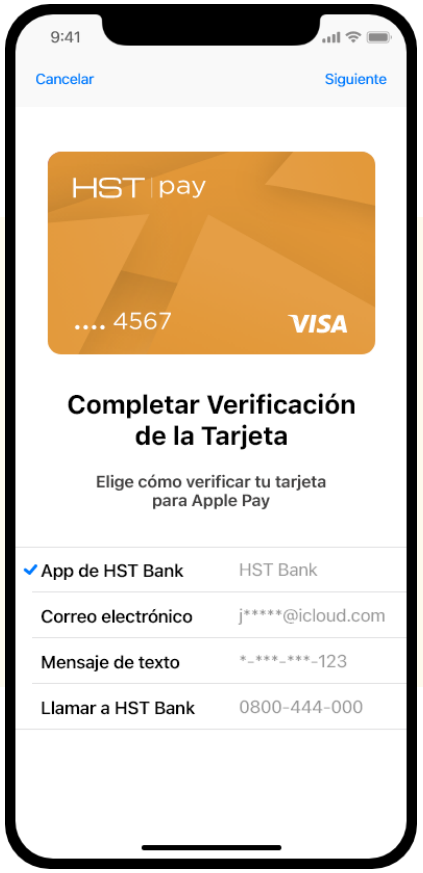
2 Emisor llama al API ActivateToken (DNITHE302101556961998459)

Emisor

Al digitalizar la tarjeta...

Flujo Verde





Flujo Amarillo

GLOSARIO Apple

Términos y nomenclaturas utilizadas por Apple en sus documentos, especificaciones y requisitos.

DPAN

Identificador del dispositivo asociado al PAN definido por la marca

FPAN

Identificador de la tarjeta definido por la marca (PANRefID)

PAN

Número de la tarjeta

PNO

La marca/franquicia

E2E

End2End

OTP

One Time Password

T&Cs

Términos & Condiciones

UI

Interface del usuario

SEID

Secure Element ID – ID de dispositivo iOS

HST

MAKING ELECTRONIC TRANSACTIONS **SECURE**